



POLITECNICO DI BARI

**DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE
LAUREA MAGISTRALE IN INGEGNERIA DELL'AUTOMAZIONE**

Embedded control

Prof. Luca De Cicco

Title:

Open-Loop Axis Control for 3D Printer

Student:

Domenico D'Introno

ACADEMIC YEAR 2024-2025



Contents

1	Objectives of the Project	2
2	Selected Hardware	3
3	Design Choices	6
4	Firmware	7
4.1	Stepper Motor Control Module	7
4.2	Command Module for Stepper Motor Control	8
4.3	Timers Configuration	10
4.3.1	TIM2 – PWM Signal Generation	10
4.3.2	TIM3 – Step Counting	10
4.3.3	TIM6 – Trapezoidal Acceleration Timing	11
4.4	UART Configuration and Reception Handler	11
4.5	Main	12
5	Demonstration Video	13
6	Conclusion	14

1 Objectives of the Project

The goal of this project is to design and implement an open-loop embedded position control system simulating the functioning of a single axis of a 3D printer. The system main capability are:

- Decoding **G-code commands** received via UART communication to change the position set-point and modify acceleration and maximum velocity of the movement;
- Controlling the **linear position** of an object via a mechanical transmission driven by a stepper motor, **without position feedback**;
- Ensuring smooth acceleration and deceleration using a **trapezoidal speed profile**, improving precision and minimizing mechanical stress.

2 Selected Hardware

The hardware components selected for this project are:

- **STM32F446RE Microcontroller (Nucleo board):** Executes control logic and manages peripherals;

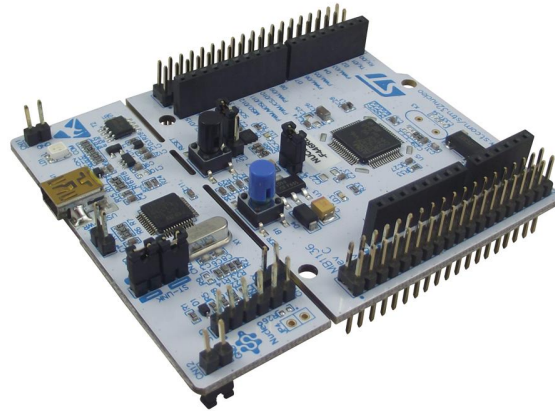


Figure 1: STM32F446RE Microcontroller

- **Nema 17 Stepper Motor:** Provides precise motion to actuate the system;

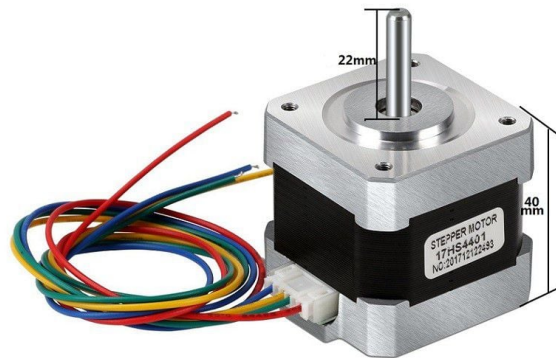


Figure 2: Nema 17 Stepper Motor

- **A4988 Stepper Driver:** Converts logic-level signals into motor drive signals;

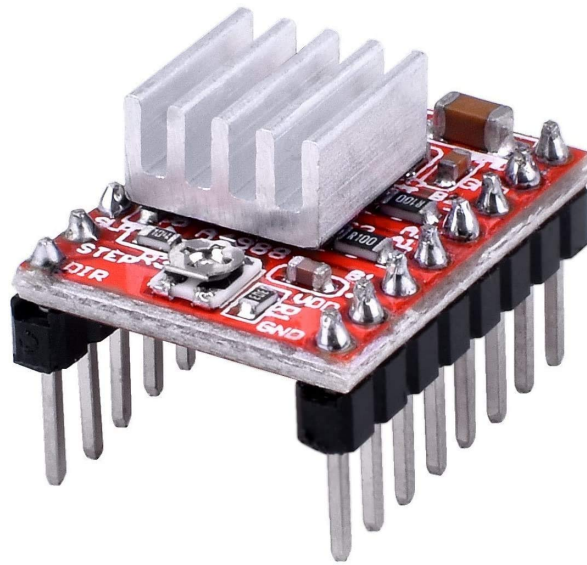


Figure 3: A4988 Stepper Driver

- **Pulley and Belt Mechanism:** Translates motor rotation into linear movement;



Figure 4: Pulley and Belt Mechanism

- **Wooden Frame:** Provides structural support for the physical setup.

The described hardware components have been mounted following the electrical scheme in fig. 5, using jumpwires and a breadboard.

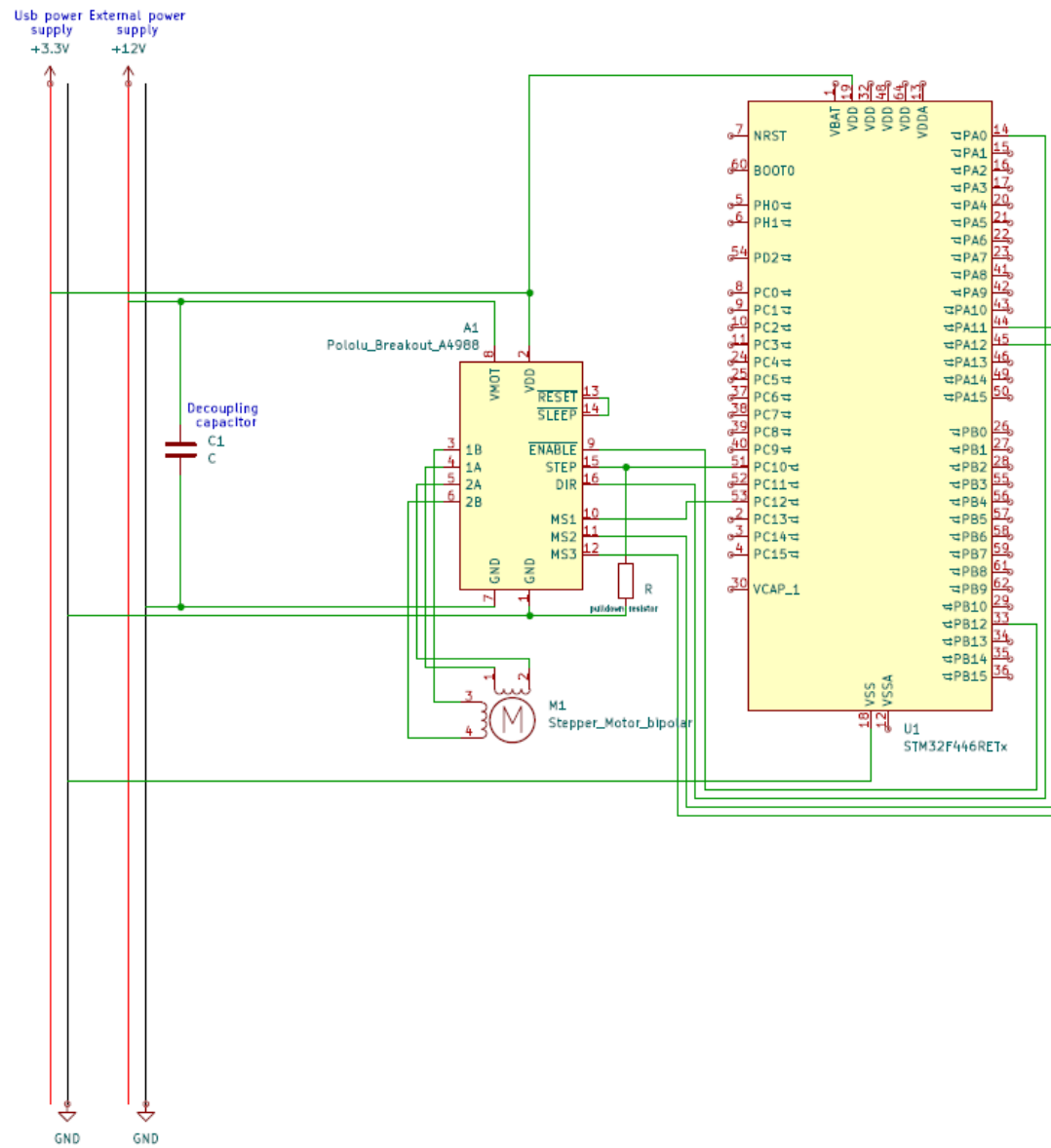


Figure 5: Kicad hardware electrical scheme

3 Design Choices

The idea of an **open-loop position control** system comes from real-world applications of this technology in low-cost 3D printers.

Using **UART communication**, the user interacts with the machine by sending commands via a terminal. These commands follow the **G-code** format and allow the user to change the position of the extruder as well as set the speed and acceleration of the movement. The commands, are processed into the UART 'receive complete' handler, which parses, identifies and executes the commands.

Since no position feedback is available, the precision of the stepper motor becomes a critical factor. This is achieved through the use of **chained timers**: one timer generates the PWM signal that drives the motor, while another counts the steps. By estimating the linear displacement of each step, a position setpoint can be converted into a specific number of steps required to reach the target. Also, speed and acceleration, from a control point of view are respectively the number of pulses generated per second and the pulses rate increase per second.

In this **sensor-less** setup, avoiding step loss is essential to maintain accuracy. For this reason, a smooth speed profile—specifically a **trapezoidal velocity profile**—is used during motion. This profile is automatically computed based on the required displacement, the maximum speed and acceleration values defined by the user. The resulting speed profile is pre-computed and stored in a **look-up table** before the movement begins. A third timer then updates the motor speed in real-time by accessing the table during motion, ensuring smooth acceleration and preserving positional accuracy. Thanks to the 'offline' computation of the LUT, timer handler can update the motor speed fastly without spending too much time in the handler.

The whole system can be considered **interrupt-driven**, since the whole code resides into interrupt handlers and the `while(1)` section is empty.

The firmware design relies on a modular and abstracted architecture, built around custom libraries **stepper.c** and **command.c**. These components encapsulate the low-level implementation details of motor control and command parsing, respectively, allowing for cleaner and more maintainable code. By separating functionality into distinct modules, the firmware becomes easier to extend and debug. This structure also promotes reusability and improves readability, enabling a clearer distinction between hardware interaction and application logic.

4 Firmware

This section describes the firmware developed for the control of the stepper motor. The code, implemented using STM32 HAL libraries, includes the definition of libraries modules, timers configuration, the communication and the control logic.

4.1 Stepper Motor Control Module

This C module defines a library for controlling a stepper motor using an STM32 microcontroller. The library provides an interface to manage PWM signals, motor direction, microstepping configuration, acceleration and speed, using hardware peripherals such as timers and GPIO ports.

Constants and Includes

- `#define clk_freq 84000000`
Defines the clock frequency used for speed calculations.
- `#include "stepper.h"`
Includes the header file containing the **Stepper** structure and enumeration definitions.

Stepper_Init

Initializes the **Stepper** structure by assigning GPIO ports and pins, the timer handle, the distance per step, and the default acceleration. The motor is disabled by default.

Stepper_Enable

Enables the motor driver by setting the enable pin low (active-low logic).

Stepper_Disable

Disables the motor driver by setting the enable pin high.

Stepper_Start

Resets speed to zero, then starts the PWM timer that pulses the motor and the secondary timer used for handling acceleration via interrupts.

Stepper_Stop

Stops both the PWM signal and the acceleration timer interrupt, effectively halting the motor.

Stepper_ToggleDirection

Toggles the rotation direction of the motor by inverting the state of the direction pin.

Stepper_SetDirection

Sets the rotation direction explicitly using a **Direction** enum parameter.

Stepper_SetMicroStep

Configures the microstepping mode (e.g., full, half, quarter, eighth, or sixteenth step) by setting the MS1, MS2, and MS3 pins accordingly. The function accepts a **MicrosteppingMode** enum parameter.

Stepper_SetSpeed

Sets the motor speed by updating the PWM timer's ARR register. The command assigns a speed in mm/s, computed using the clock frequency, distance per step and desired speed. The timer counter is reset to avoid inconsistencies.

Having the `stepDist` parameter, whose value describes the covered linear distance in mm for one step, and the desired speed in mm/s in the `speed` parameter, the number of steps per second can be computed:

$$stepPerSecond = \frac{speed}{stepDist}$$

`stepPerSecond` represents basically the frequency of the PMW pulses in order to move the motor at the desired speed and its inverse is the counting period of the PWM timer.

To set this counting period, having fixed PSC value, ARR must be:

$$ARR = \frac{f_{clk}}{stepPerSecond(PSC + 1)} - 1$$

Where $f_{clk} = 84$ MHz is the frequency of the clock that feeds the timer. In this case, indeed, TIM2 is connected to APB1.

Stepper_SpeedUp

Increases the current motor speed by a given delta value. This internally calls `Stepper_SetSpeed`. This command can be used to implement acceleration in some different design logic.

4.2 Command Module for Stepper Motor Control

The `command.c` module implements a command parser and executor for controlling a stepper motor system via standard G-code and M-code commands. It communicates through UART and allows parsing, interpreting, and executing commands for the real-time control of motion, speed, and acceleration parameters.

Included Headers and External Variables

- `#include "command.h", #include "stepper.h"`: Include the definitions of the command and stepper structures.
- `speed_table, speed_table_size, last_position`: External variables used for generating and storing the velocity profile.

Command_Init

Initializes the `Command` structure by linking the stepper, timer, and UART handles. It also sets default speeds and accelerations.

Command_Parse

Parses a string formatted as a G-code/M-code command (e.g., `G0 X100`), extracting the command type, number, flag type and flag number, through which the desired action is specified.

Command_Clear

Clears the current command structure fields and resets the input buffer to prepare for a new command.

Command_Execute

Executes the parsed command by calling the corresponding handler function. If the command is not recognized, an error message is sent over UART.

Command_G0

Implements the G0 command. It calculates the number of steps required to reach a new position, allocates a velocity profile, and starts the stepper motor using the appropriate direction. It also performs validation and memory management.

Command_M203

Handles the M203 command, which sets travel speed depending on the provided flag.

Command_M204

Handles the M204 command to update the acceleration values, based on the provided flag.

Command_M205

Implements the M205 command to change the rotation direction of the motor. A flag of 1 sets clockwise rotation, while 2 sets counter-clockwise. This command is not used during normal operations, since motor rotation direction is automatically computed based on the setpoint to be reached. However this command has been useful for debug.

Speed Profile Computation

The function `ComputeSpeedProfile` generates a **trapezoidal speed profile** based on classical kinematic equations, ensuring smooth acceleration and deceleration of the stepper motor. The profile is computed using acceleration a and step displacement $stepDist$.

To reach a given **maximum speed** $v_{\max}$, the number of steps required is calculated using the equation:

$$s_{\text{acc}} = \frac{v_{\max}^2}{2a \cdot stepDist}$$

where s_{acc} is the number of steps required during the acceleration phase. If the total number of steps N is too small to reach $v_{\max}$ before deceleration, the function dynamically computes the maximum reachable speed by inverting the same formula:

$$v_{\max} = \sqrt{2a \cdot stepDist \cdot \frac{N}{2}}$$

The final profile is divided into three phases:

- **Acceleration phase:** $v(i) = \sqrt{2a \cdot stepDist \cdot i}$
- **Constant velocity phase:** $v(i) = v_{\max}$
- **Deceleration phase:** $v(i) = \sqrt{2a \cdot stepDist \cdot (N - i)}$

This approach guarantees smooth transitions between motion phases while ensuring that the motor performs every step without skipping, even in the absence of position feedback.

4.3 Timers Configuration

In this system, three hardware timers (TIM2, TIM3, and TIM6) are configured to coordinate the control of a stepper motor using a trapezoidal velocity profile.

To achieve accurate stepper motor control, the firmware makes use of a chained timers configuration, a powerful hardware feature of STM32 microcontrollers. In this setup, **TIM2** is configured as the **master**, generating a PWM signal used to drive the motor. **TIM3** acts as the **slave** in external clock mode, using the PWM signal from TIM2 as its clock input. This is done by connecting TIM2's update event to TIM3 via the internal trigger line ITR1. As a result, each time TIM2 produces a step pulse, TIM3 increments its counter. The following figure extracts shows the internal trigger connections: it can be seen that TIM2 and TIM3 are connected through ITR1 line

Slave TIM	ITR0 (TS = 000)	ITR1 (TS = 001)	ITR2 (TS = 010)	ITR3 (TS = 011)
TIM2				
TIM3		TIM1		
TIM4		TIM1		
TIM5				

Figure 6: Timers internal trigger connection table

This chained configuration ensures precise step counting in hardware, without the need for generating interrupt every step. When TIM3 reaches the target step count, it triggers an interrupt that stops the motor.

Each timer serves a specific function:

4.3.1 TIM2 – PWM Signal Generation

TIM2 is responsible for generating the PWM signal required to drive the stepper motor. It operates in PWM mode (PWM1) and outputs the signal on pin PA0 (TIM2_CH1) that is electrically connected to the A4988 driver STEP pin. The speed of the motor is regulated by dynamically updating the timer's auto-reload value based on the computed LUT. The configuration is as follows:

- **Prescaler:** 0 (maximum timer resolution)
- **Period:** initialized to 839999 (with an 84 MHz clock, produces a 100 Hz PWM signal), this parameter is changed at run-time by the `Stepper_SetSpeed` function to obtain a desired speed.
- **Pulse:** 10000 this value guarantees consistent duty cycles across all allowed speed interval.
- **Output Mode:** PWM1
- **Clock Source:** Internal
- **Master Mode:** Enabled (TIM_TRGO_UPDATE)

4.3.2 TIM3 – Step Counting

TIM3 is used to count the number of steps the motor has executed. It is configured in slave mode, synchronized with TIM2 via the internal trigger ITR1. Upon reaching the defined number of steps, TIM3 triggers an interrupt that stops the motor and disables the driver.

- **Prescaler:** 0
- **Period:** this parameter is changed at run-time by the `GO_command` function to obtain the desired number of steps.
- **Slave Mode:** External Clock Mode 1 (TIM_SLAVEMODE_EXTERNAL1)

- **Input Trigger:** TIM_TS_ITR1

In the interrupt handler (`HAL_TIM_PeriodElapsedCallback`), which is executed when the number of required steps are performed, the motor is stopped, the driver is disabled, and the trapezoidal speed table is deallocated.

4.3.3 TIM6 – Trapezoidal Acceleration Timing

TIM6 acts as a periodic timer to regulate the acceleration profile of the motor. It generates periodic interrupts (approximately every 0.5 ms) to update the motor speed by accessing the LUT table.

- **Prescaler:** 499
- **Period:** 8399 (results in a 0.5 ms interval at 84 MHz)
- **Counter Mode:** Up
- **Trigger Output:** Reset

During each interrupt, the system reads the current step index from TIM3 and sets the motor speed to the corresponding value in the speed table.

This configuration ensures synchronized, accurate stepper motor control with acceleration and deceleration capabilities.

4.4 UART Configuration and Reception Handler

The UART communication is used to receive commands and to send log data and error messages through the serial terminal Putty. In this project, **USART2** is used and it is configured with the following parameters:

- **Baud rate:** 115200 bps
- **Word length:** 8 bits
- **Stop bits:** 1
- **Parity:** None
- **Mode:** TX/RX (transmit and receive)
- **Oversampling:** 16

The hardware initialization is performed in `HAL_UART_MspInit()`, which enables the USART2 clock and configures GPIO pins PA2 and PA3 in alternate function mode for TX and RX respectively. Interrupts are also enabled.

The reception of UART data is handled asynchronously using interrupts. The function `HAL_UART_RxCpltCallback()` is triggered whenever a byte is received. The firmware uses this mechanism to construct a command buffer, processing complete commands terminated by the `\r` character.

The interrupt handler performs the following operations:

1. Appends the received byte to the buffer `rx_data`.
2. If the received byte is a carriage return (`\r`), the full command is parsed using `Command_Parse()` and executed via `Command_Execute()`.

3. The buffer is cleared and reset using `memset()` and `Command_Clear()`.
4. The UART is re-armed to receive the next byte using `HAL_UART_Receive_IT()`.

This interrupt-driven approach ensures a responsive and non-blocking communication mechanism, ideal for command-based control interfaces.

4.5 Main

The **main.c** file serves as the entry point of the firmware and contains the initialization of the core components involved in the control loop. In particular, the user-defined sections are responsible for setting up and configuring the **stepper motor** and the **command interface**.

The function `Stepper_Init()` initializes the stepper motor driver structure, associating it with hardware resources such as timers and GPIO pins. The stepping mode is then configured using `Stepper_SetMicroStep()`, which selects the desired resolution (e.g., quarter-step mode).

The command parser is initialized through `Command_Init()`, linking it to the stepper motor, a timer for step counting, and a UART peripheral for receiving commands from a terminal. UART reception is then enabled in interrupt mode via `HAL_UART_Receive_IT()`, allowing asynchronous communication with the user.

Timer 3, acting as a slave counter, is explicitly started with interrupts enabled using `HAL_TIM_Base_Start_IT()`, after clearing its update flag to ensure clean startup conditions.

Notably, the **main loop** (`while(1)`) remains empty, as the firmware is designed to be **interrupt-driven**. All the logic is handled asynchronously through hardware interrupts.

5 Demonstration Video

The linked demonstration video shows:

- UART command control;
- Stepper motor movement;
- SWV (Serial Wire Viewer) real-time tracing of the speed variable.

Link: <https://youtu.be/1Xp8fLpwAnw>

6 Conclusion

The project successfully implements a precise motion control system for a 3D printer axis using an STM32 microcontroller. This result has been reached even if no feedback sensor is used, by increasing software complexity introducing clever speed handling through trapezoidal profiles. The architecture is efficient, modular, and can be extended to multi-axis systems or upgraded with closed-loop control strategies.